Built-in Modules

1. System and OS Interaction Modules

1.1 `os` – Operating System Interface

The `os` module provides functions to interact with the operating system.

Common Functions:

Function	Description
`os.getcwd()`	Returns the current working directory
`os.chdir(path)`	Changes the current working directory
`os.listdir(path)`	Returns a list of files and folders in a directory
`os.mkdir(name)`	Creates a new directory
`os.remove(file)`	Deletes a specified file
`os.rmdir(dir)`	Removes an empty directory
`os.path.exists(path)`	Checks if a path exists

Example:

```python
import os

print("Current Directory:", os.getcwd())
if not os.path.exists("example_folder"):
    os.mkdir("example_folder")
```

1.2 `sys` – System-Specific Parameters and Functions

The `sys` module provides access to interpreter-specific functions and variables.

Common Functions:

Function	Description
`sys.argv`	List of command-line arguments
`sys.exit()`	Exits the program
`sys.path`	List of paths for module search
`sys.version`	Returns the Python version

Example:

```python
import sys

print("Python Version:", sys.version)
```

1.3 `platform` – System and Platform Information

Provides details about the system, OS, and hardware.

Common Functions:

Function	Description
`platform.system()`	Returns OS name (e.g., Windows, Linux)
`platform.release()`	OS release version
`platform.processor()`	Returns processor type

Example:

```python
import platform

print(platform.system(), platform.release(),
platform.processor())
```

2. `json` – JSON Encoding and Decoding

The `json` module allows encoding Python objects into JSON format and decoding JSON back to Python.

Functions:

Function	Description
`json.dumps(obj)`	Converts Python object to JSON string
`json.loads(json_str)`	Converts JSON string to Python object
`json.dump(obj, file)`	Writes JSON object to a file
`json.load(file)`	Reads JSON data from a file

Example:

```python
import json

data = {"name": "Alice", "age": 30}
json_str = json.dumps(data)
print(json_str)

parsed = json.loads(json_str)
print(parsed["name"])
```

3. Mathematics and Randomness Modules

3.1 `math` – Mathematical Functions

Provides mathematical operations and constants.

Constants:

Constant	Description
`math.pi`	π = 3.14159...
`math.e`	Euler's number ≈ 2.718

Functions:

Function	Description
`math.sqrt(x)`	Returns the square root of x
`math.pow(x, y)`	x raised to the power y (x^y)
`math.ceil(x)`	Smallest integer ≥ x
`math.floor(x)`	Largest integer ≤ x
`math.fabs(x)`	Absolute value of x
`math.factorial(x)`	Factorial of x (x!)
`math.gcd(x, y)`	Greatest common divisor
`math.log(x, base)`	Logarithm of x to the given base
`math.sin(x)`	Sine of x (x in radians)
`math.cos(x)`	Cosine of x
`math.tan(x)`	Tangent of x
`math.degrees(x)`	Convert radians to degrees
`math.radians(x)`	Convert degrees to radians

Example:

```python
import math
print(math.sqrt(25))
print(math.factorial(5))
print(math.sin(math.pi / 2))
```

3.2 `random` – Generate Random Numbers

Used for random selections, number generation, and shuffling.

Functions:

Function	Description
`random.random()`	Returns a float in the range [0.0, 1.0)
`random.randint(a, b)`	Returns random integer between a and b (inclusive)
`random.uniform(a, b)`	Returns a float between a and b
`random.choice(seq)`	Returns a random element from a non-empty sequence
`random.choices(seq,k=n)`	Returns a list of `k` random elements from `seq`
`random.shuffle(list)`	Shuffles the list in place
`random.seed(n)`	Sets the seed for reproducibility

Example:

```python
import random

print(random.randint(1, 100))
print(random.choice(['apple', 'banana', 'cherry']))
```

4. Internet Data Handling Modules

4.1 `urllib` – URL Handling

Used for fetching data from URLs.

Common Use:

```python
from urllib import request

response = request.urlopen('https://www.example.com')
html = response.read().decode('utf-8')
print(html)
```

4.2 `http.client` – HTTP Protocol Support

Used to make HTTP requests.

Example:

```python
from http.client import HTTPConnection

conn = HTTPConnection("www.example.com")
conn.request("GET", "/")
response = conn.getresponse()
print(response.status, response.reason)
```

5. Data Structures and Iteration Utilities

5.1 `collections` – Specialized Data Structures

Provides alternatives to built-in types for performance and convenience.

Key Classes:

Class	Description
`Counter`	Counts frequency of elements
`defaultdict`	Dictionary with default values
`namedtuple`	Tuple with named fields
`deque`	Double-ended queue for fast appends/pops
`OrderedDict` (Py < 3.7)	Dictionary that maintains insertion order

Example:

```python
from collections import Counter, defaultdict
```

```
data = ['a', 'b', 'a', 'c']
counter = Counter(data)
print(counter)

dd = defaultdict(int)
dd['missing'] += 1
print(dd['missing'])  # Output: 1
```

5.2 `itertools` – Efficient Iteration Utilities

Provides tools for creating iterators for efficient looping.

Common Functions:

Function	Description
`count(start, step)`	Infinite sequence: start, start+step, ...
`cycle(iterable)`	Repeats the iterable infinitely
`repeat(elem, n)`	Repeats element n times
`accumulate(iterable)`	Cumulative sums
`chain(a, b)`	Combines multiple iterables
`combinations(iter, r)`	r-length tuples, combinations without replacement
`permutations(iter, r)`	r-length tuples, all possible orderings
`product(iter1, iter2)`	Cartesian product

Example:

```python
import itertools

print(list(itertools.count(10, 2)))  # Infinite, use with break
print(list(itertools.combinations('ABCD', 2)))
```

Summary

Module	Key Use
`os`	OS-level file/directory operations
`sys`	Interpreter control, arguments, path
`platform`	System and hardware information
`json`	Encode/decode JSON data
`math`	Mathematical operations and constants
`random`	Random number generation and selections
`urllib`	Web data fetching
`http.client`	Low-level HTTP requests
`collections`	Advanced container types
`itertools`	Efficient looping and iteration tools